

/dirs

Manage directory/file permission lists: /dirs add|remove allow|deny [--global] | show | help

Overview

/dirs manages the user-defined allowlist, denylist, and allowed-roots lists consumed by the file-path permission guards. Allowlist patterns auto-approve matching file operations before the user permission prompt is shown; denylist patterns always reject them, and deny overrides allow. roots entries extend the set of directories treated as valid roots by the path-escape check. Changes are persisted immediately: project scope writes `.ragent/ragent.json`, global scope writes `~/.config/ragent/ragent.json`.

Syntax

```
/dirs add allow <pattern> [--global]
/dirs add deny <pattern> [--global]
/dirs add roots <path> [--global]
/dirs remove allow <pattern> [--global]
/dirs remove deny <pattern> [--global]
/dirs remove roots <path> [--global]
/dirs show
/dirs help
```

Notes:

- `--help`, `-h`, and the empty form also print the help page.
- `--global` may trail the pattern; it is trimmed off the pattern text and redirects the write to `~/.config/ragent/ragent.json`.
- Pattern matching uses glob syntax: `*` matches any characters except `/`, `**` matches any characters including `/`, `?` matches a single character, and `[abc]` matches one character from the set.
- When no roots entries exist, only the session working directory is treated as a valid root for path-escape checking.

Options / Subcommands

Form	Description
/dirs add allow <pattern> [--global]	Add a glob pattern to the allowlist. File operations matching the pattern are automatically allowed.
/dirs add deny <pattern> [--global]	Add a glob pattern to the denylist. File operations matching the pattern are automatically denied. Deny overrides allow.
/dirs add roots <path> [--global]	Add a directory to the allowed roots used by path-escape checking.
/dirs remove allow <pattern> [--global]	Remove a pattern from the allowlist.
/dirs remove deny <pattern> [--global]	Remove a pattern from the denylist.
/dirs remove roots <path> [--global]	Remove a directory from the allowed roots.
/dirs show	Print the full list report (see Output).

Form	Description
<code>/dirs help</code>	Print the help page.

Examples

Auto-approve all Rust sources under `src/`:

```
/dirs add allow src/**/*.rs
```

Deny anything under a secrets tree (denial wins over a matching allow):

```
/dirs add deny secrets/**
```

Treat a sibling project directory as a valid root for path-escape checking:

```
/dirs add roots ../other-project
```

Remove a deny pattern from the user-global config:

```
/dirs remove deny secrets/** --global
```

Inspect every list the path guards consult:

```
/dirs show
```

Unknown list type prints the correction hint:

```
/dirs add foo src/**
```

Output

`/dirs show` prints five sections in this order. Empty user lists print `*(empty)*`.

1. Built-in Allowlist - auto-approved by default.
2. User Allowlist - your auto-approve patterns.
3. Built-in Denylist - always rejected by default.
4. User Denylist - your rejection patterns (deny overrides allow).
5. Allowed Roots - extra valid root directories for path-escape checking.

Message formats:

- Successful allow/deny add: File operations matching X will be automatically allowed/denied
- Successful roots add/remove: Path escape checking will treat X as a valid root directory
- Remove of an entry that is not present: the same [warn] ... was not in the {scope} allowlist/denylist wording used by /bash
- Backend failure: [err] Error: {e}
- Empty pattern: the usage line.
- Unknown list type (anything other than allow, deny, or roots): Use allow, deny, or roots
- Unknown subcommand: Run `/dirs help`

Patterns are checked before the user permission prompt, so an allowlist match suppresses the prompt and a denylist match short-circuits it.

Related

- Bash command lists: `/bash` (same add/remove/show model and `--global` scoping).
- Permission rules, allow/deny/ask ordering, and YOLO mode: `docs/howtos/permissions.md`.
- Config files written: `.ragent/ragent.json` (project), `~/.config/ragent/ragent.json` (global).